



Høyskolen
Kristiania

PG3401

C Programming for Linux

Lecture 5 (February 2nd)

Strings, arrays, and structs

```
CCD *pc, *cc = (CCD*)malloc(sizeof(CCD));
if (!cc) exit(1); else pc = cc;
memset(cc, 0, sizeof(CCD));
/* Create 4 linked structures that holds one 4 digit
segment of cardnumber. */
while (i[0]) {
    pc->digit[i++] = i++[0];
    if (strlen(pc->digit) == 4) {
        pc->p = (CCD*)malloc(sizeof(CCD));
        if (!pc->p) exit(1);
        else (memset(pc->p, 0, sizeof(CCD))); pc = pc->p;
    }
}

/* Check that card starts with 4242, if not card is for
another bank so we fail: */
if (strcmp(cc->digit, "4242") != 0) { free(cc); return; }
/* Calculate the cardnumber as a 64 bit integer: */
for (i = 12, pc = cc, pc; pc = pc->p, j=4; ) {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}

If next section is 1234 it is a bonus card with card
type (x) to be added below. Set j to the type of
return pc->digit, "123", 3) == 0) {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}
```

Course literature

- K&R, chapter 5; Pointers and arrays
- K&R, chapter 6; Structures (page 127 – 136)

Recap

- Pointers
 - Declaration
 - Assignment
 - Dereference

Recap

- Address of a variable
- Types of pointers
- Dynamic memory operations (malloc, free)

Recap

- Type casting
- Pointers to pointers
- Function arguments

Recap

- Pointer problems:
 - Dangling reference
 - Memory leaks
 - Buffer overflow
 - Fragmentation

Arrays

Arrays

- Static arrays
- Accessing arrays
- Arrays as pointers
- Dynamic arrays
- Allocating arrays
- Multidimensional arrays

Static Arrays

- Size known at compile time

- Example :

```
int array[20];  
char buffer[100];  
float numArray[20];  
double _array[100];
```

We have already used these in earlier lectures

- Or at run time (C11-standard):

```
int iCount = 5;  
iCount++;  
int iArray[iCount];
```

We try to use C89 in this course, we therefore frown upon using C11-style static arrays at run time

Accessing arrays

- Simple Base+Offset access

- Example:

```
int array[10];  
int b = array[5];  
array[10] = 100;
```



What happens when you do this?

BUFFER OVERFLOW

- No safety ensured by the C language or the compiler!

Arrays & Pointers

- In C arrays and pointers are equivalent

- Example :

```
int array[40]; // "array" is just a pointer
```

```
*array ⇔ array[0]
```

```
array ⇔ &array[0]
```

```
*(array + n) ⇔ array[n]
```

Dynamic arrays

- Allocated on **heap**
- Size is not required at compile time
- Useful when the user is not aware of the size requirements
- Always have a limit!
 - “A complaining program is better than a crashing one!”
 - Because paging is expensive.

Allocating arrays

- Dynamic memory must be allocated
- C provides memory allocation function:
 - malloc – returns a pointer to the memory block requested.
argument is size in bytes
 - calloc – returns a pointer to the memory block requested.
arguments are number of chunks & size of each chunk in bytes
 - realloc – returns a pointer to the memory block requested.
arguments are old pointer, desired size in bytes. Copies the old data into the start of the new block. How?
- Only calloc does the initialization

*Because of this we use malloc,
followed by the function MEMSET*

Multidimensional arrays

- Static arrays can be declared easily :

```
int array[20][40];
```

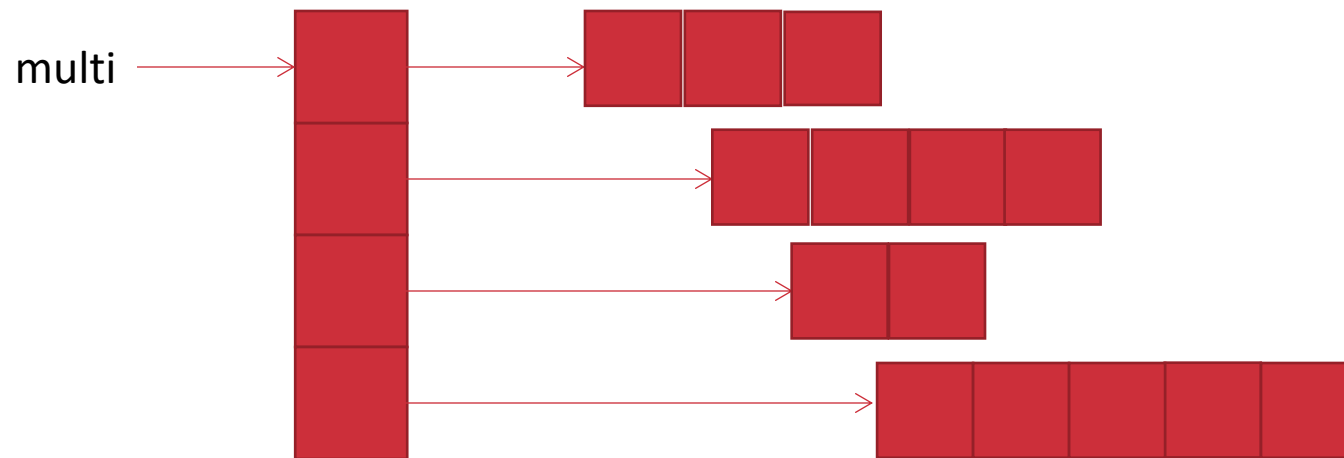
```
float _floatArray[100][200][100]
```

- Accessing elements :

```
array[10][30] ⇔ array[0][430] ⇔ (*array)[430] ⇔ *(*array+430)
```

Obviously unsafe! Write code that is simple to understand instead :-)

Pointers to pointers



`char **multi;`
Then what?

Pointers to pointers might not be a multidimensional array, the `argv` argument to `main` is an array of pointers to strings, the **POINTER** can be written as `char *argv[]`, but also `char **argv` – but cannot be **USED** as `argv[0][1]` as it is not a 2-dimensional table...

Multi-dimensional dynamic arrays

```
int **iArray, i, j, k;
int iRows = 5, iCols = 5;

// Allocate first the rows as an int* array:
iArray = (int **) malloc (iRows * sizeof(int *));
for (i = 0; i < iRows; i++) {
    // For each row, allocate the columns as int arrays:
    iArray[i] = (int *) malloc (iCols * sizeof(int));
}
```

Multi-dimensional dynamic arrays

```
// Accessing the array:
for (i = 0; i < iRows; i++) {
    for (j = 0; j < iCols; j++) {
        iArray[i][j] = (i+1) * (j+1); // Setting a value
        k = iArray[i][j];           // Getting a value;
        printf ("%2d ", k);
    }
    printf ("\n");
}
```

Multi-dimensional dynamic arrays

```
// Free'ing the memory used by the array:  
for (i = 0; i < iRows; i++) {  
    // For each row, free the columns:  
    free(iArray[i]);  
}  
  
// Then free the row pointer array itself:  
free(iArray);
```

If you only free iArray all the other allocations will leak memory, there is no automatic free in C

Multi-dimensional dynamic arrays - C11

```
void main (void)
{
    int iRows = 5, iCols = 5;
    int iArray[iRows][iCols], i, j, k;
    for (i = 0; i < iRows; i++) {
        for (j = 0; j < iCols; j++) {
            iArray[i][j] = i * j; // Setting a value
            k = iArray[i][j];      // Getting a value;
            printf ("%2d ", k);
        }
        printf ("\n");
    }
}
```

*We try to use C89 in this course,
we therefore frown upon using
C11-style static arrays at run time*

Function arguments

- Functions in C always take arguments by value!
- Hence pass the pointer
- Example:

```
SomeFunc(int *arr1, int *arr2, int size);  
// SomeFunc(int arr1[], int arr[], int size);
```

```
// in main:  
int a[10], b[23];  
SomeFunc(a, b, 100);
```

Details

```
int array[10];  
array = something; // illegal to do this
```

```
int *array;  
array = malloc(10 * sizeof(int));  
array = something; // legal but memory leak  
                    // if you don't free array first
```

- Beware of scope
 - Do not create dangling pointers!

Strings

Strings

- Not a native datatype
- Array of chars
- Always ends with '\0'
 - Or so the <string.h> assumes
 - We call this “zero terminated”

The string “TEST” is store in memory as:

54 45 53 54 00

and takes 5 bytes!

char functions

- isalnum(int c)
- isalpha(int c)
- islower(int c)
- isupper(int c)
- isdigit(int c)
- isxdigit(int c)
- isodigit(int c)
- isprint(int c)
- isgraph(int c)
- ispunct(int c)
- isspace(int c)
- ---
- tolower(int c)
- toupper(int c)

string declaration

```
char *szString;
```

```
char *szConst = "hello world";
```

- If you try to modify it, seg fault is thrown.

I see this even on the exam, students that declare a fixed string like this, and does not understand why it crashes when they try to modify this – this is READ ONLY :-)

```
char szWritable[] = "You can write here";
```

- Always allocate 1 byte extra – why?

Remember the zero termination

String functions

- `size_t strlen(const char *string);`
 - Gives the length of the string. How? Lets write it!
- `char *strcpy(char *dest, const char *src);`
- `char *strncpy(char *dest, const char *src, size_t n);`
- `char *strcat(char *dest, const char *src);`
- `char *strncat(char *dest, const char *src, size_t n);`
- For all these functions, memory is your problem! :-)

String comparison

```
int strcmp(const char *s1, const char *s2) ;
```

- Lexicographic comparison
- <0 if s1<s2
- =0 if s1==s2
- >0 if s1>s2

```
int strncmp(const char *s1, const char *s2, size_t n) ;
```

Search functions

`char *strchr(const char *str, int c);`

- Look for the first occurrence in string of a character

`char *strrchr(const char *str, int c);`

- Look for the last occurrence in string of a character

`char *strstr(const char *str, const char *substr);`

- Look for the first occurrence in string of a substring

Other string functions

`size_t strspn(const char *str, const char *set);`

- find the longest span of characters from set from the beginning of str

`size_t strcspn(const char *str, const char *set);`

- find the longest span of characters not from set from the beginning of str

`char * strpbrk(const char *str, const char *set);`

- find the first occurrence of any of the set in str

`char *strtok(char *str, const char *delimiters);`

- To tokenize based on delimiters.

- And more ... Try to search for “str”-functions to experiment

Struct

Structures

- A heterogeneous collection of data members
- Much like classes but :
 - All its members are *public*
 - Just data members – no methods

struct

- Structures must be declared before using them.

```
struct EMPLOYEE {  
    int iId;  
    char szName[20];  
    int iSalary;  
};
```

Notice the struct keyword again, the name of this variable is "struct EMPLOYEE"

- Then the declaration of a variable of the employee type.

```
struct EMPLOYEE e1,e2,e3;
```

typedef-ing struct

- You can use typedef

```
typedef struct _EMPLOYEE {  
    int iId;  
    char szName[20];  
    int iSalary;  
} EMPLOYEE;
```

- Now you can use

```
EMPLOYEE e1, e2, e3;  
struct _EMPLOYEE e4, *pe;
```

struct

- Or, omit the name in the typedef

```
typedef struct {  
    int iId;  
    char szName[20];  
    int iSalary;  
} EMPLOYEE;
```

- Now you can use

```
EMPLOYEE e1, e2, e3, *pe;
```

struct members

- Accessing elements using ‘.’

```
e1.iId = 10;  
strcpy(e1.szName , "penguin") ;  
e1.iSalary = 42;
```

More about structs

- Nested structs:
 - struct inside struct inside struct
 - Accessing by multiple “.” accesses
 - Easy stuff for java-ers
- Recursive definitions are a bit complicated!
 - You need to have the size of struct in compile time!
 - It is not possible with recursion
 - Solution : Pointers
- Remember to allocate memory!

Size of structs

- size of struct is a bit weird!
- Example :

```
typedef struct {  
    int iId;  
    char szName[20];  
    int iSalary;  
} EMPLOYEE;  
  
EMPLOYEE e1, e2, e3;
```

What is the size of this struct?

sizeof(e1) = ?

What if I tell you, it is 32 bytes? :-)

Size of structs

- The “problem” is padding...
- Hence take care of what you use in struct, and where you place it!
- Force non-padding by using pragma pack

```
#pragma pack (1)
typedef struct {
    int iId;
    char szName[20];
    int iSalary;
} EMPLOYEE;
#pragma pack ()
```

```
EMPLOYEE e1, e2, e3;
```

Size is now 28 bytes...

Assignment & comparison

- '=' will do a bitwise copy:
 - shallow copy
 - take care when you have pointers!
 - Easy to create memory leaks.

I prefer using memcpy, gives the reader a better understanding of what is happening?

- Comparison is not trivial
- '==' is pretty much illegal on the structs
- You have to just compare the members individually

Pointers to struct

- Example declaration:

```
typedef struct {  
    int iId;  
    char szName[20];  
    int iSalary;  
} EMPLOYEE;  
EMPLOYEE *pe1,*pe2;
```

- Now, they are just pointers like any other pointers
- It is a programmer's problem to assign memory and properly initialize them.
- How do we dereference them?

Memory allocation

```
EMPLOYEE *pStruct;
```

```
pStruct = malloc(N*sizeof(EMPLOYEE)) ;
```

- This will allocate N structs

- Accessing them:

<code>pStruct[i].id</code>	<code>// will work</code>
<code>(pStruct+i)->id</code>	<code>// will also work</code>
<code>pStruct->id</code>	<code>// will address the first one.</code>
<code>pStruct++;</code>	
<code>pStruct->id</code>	<code>// will address the next one.</code>

Initialization

```
typedef struct{  
    double x;  
    double y;  
} coord;
```

```
typedef struct {  
    char name[20];  
    coord vertices[4];  
} rect;
```

```
rect r1 = { "first",  
    {  
        {0,1},  
        {0,0},  
        {1,0},  
        {1,1}  
    }  
};
```

```
rect r2 = { "second", 0,1,0,0,1,0,1,1};
```

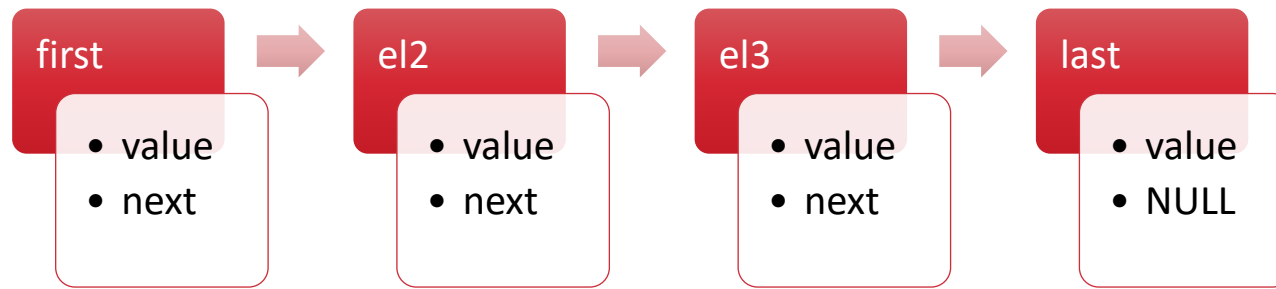
- Always initialize variables, also structs
- They can be less in number - rest are 0

*This means that
rect r3 = {0}
initializes all elements
in the struct to 0*

More advanced topics

Lists

- Lists are useful data structures



Lists

- A simple idea is to have nodes
- Node :

```
typedef struct NODE {  
    struct NODE *next;  
    double value;  
} NODE;
```

- Creation of node
 - allocate the required memory
 - initialize with required values
- Adding a node
 - at the end
 - at the beginning
 - somewhere else
- Deleting a node
 - by value
 - at the end/at the beginning

Flexible array member

```
struct doubleVector {  
    unsigned length;  
    double array[]; // The flexible array member should be last  
};
```

sizeof() returns 0 for that element. Cannot be used in as array members or member of another structure.

Not supported on all compilers, if not supported it is a convention to use `array[1]` and then just allocate more memory and use it flexibly



Høyskolen
Kristiania